



PROJECT – 6

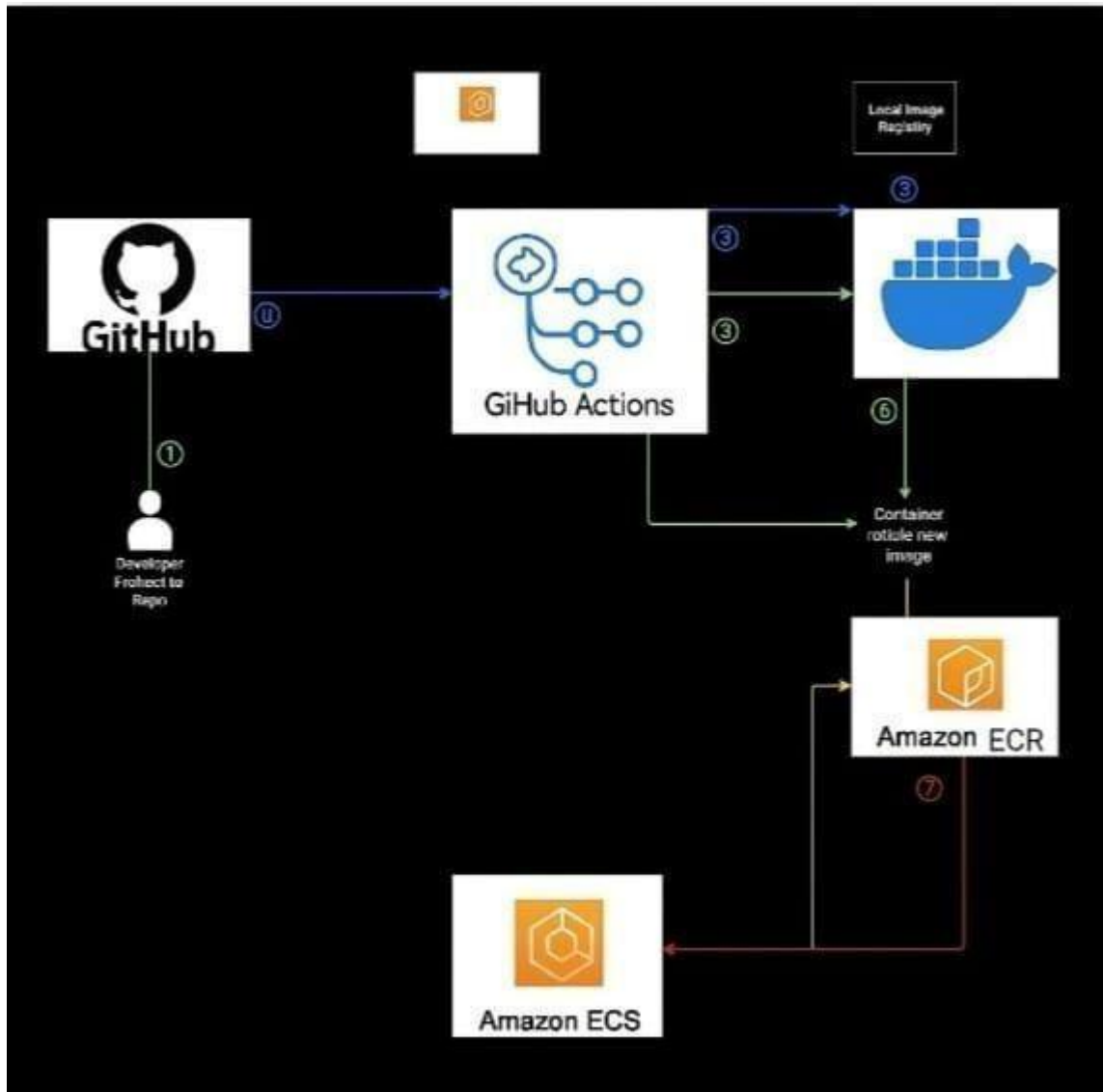
Automated CI/CD Deployment of Java Application Using GitHub Actions, Docker, Amazon ECR, and Amazon ECS

KOUJULURI VEERA BHASKARA DURGA PRASAD

Veerabhaskar90006@gmail.com

Batch No : 147

Automated CI/CD Deployment of Java Application Using GitHub Actions, Docker, Amazon ECR, and Amazon ECS



Tools & Technologies Used

- **GitHub** – Source code management
- **GitHub Actions** – CI/CD automation
- **Docker** – Application containerization
- **Amazon ECR** – Private Docker image registry
- **Amazon ECS (Fargate)** – Serverless container deployment
- **IAM** – Authentication and authorization
- **CloudWatch** – Logging and monitoring

INTRODUCTION

In today's software industry, delivering applications quickly and reliably is essential to meet business and user demands. Traditional application deployment methods involve manual steps that are time-consuming, error-prone, and difficult to maintain as systems grow in complexity. To solve these challenges, modern software development relies on **DevOps practices**, especially **Continuous Integration and Continuous Deployment (CI/CD)**, which automate the process of building, testing, and deploying applications.

This project presents the design and implementation of an **automated CI/CD pipeline for a Java-based application** using **GitHub Actions**, **Docker**, **Amazon Elastic Container Registry (ECR)**, and **Amazon Elastic Container Service (ECS)**. The main objective is to automate the complete deployment workflow, ensuring that every code change is automatically built, containerized, and deployed to the cloud without manual intervention.

When a developer pushes source code to a GitHub repository, **GitHub Actions** is triggered to execute the CI/CD pipeline. The pipeline builds the application and creates a Docker image, which packages the application along with all its required dependencies. This ensures consistency across development, testing, and production environments. The generated Docker image is then pushed to **Amazon ECR**, which acts as a secure and scalable image repository.

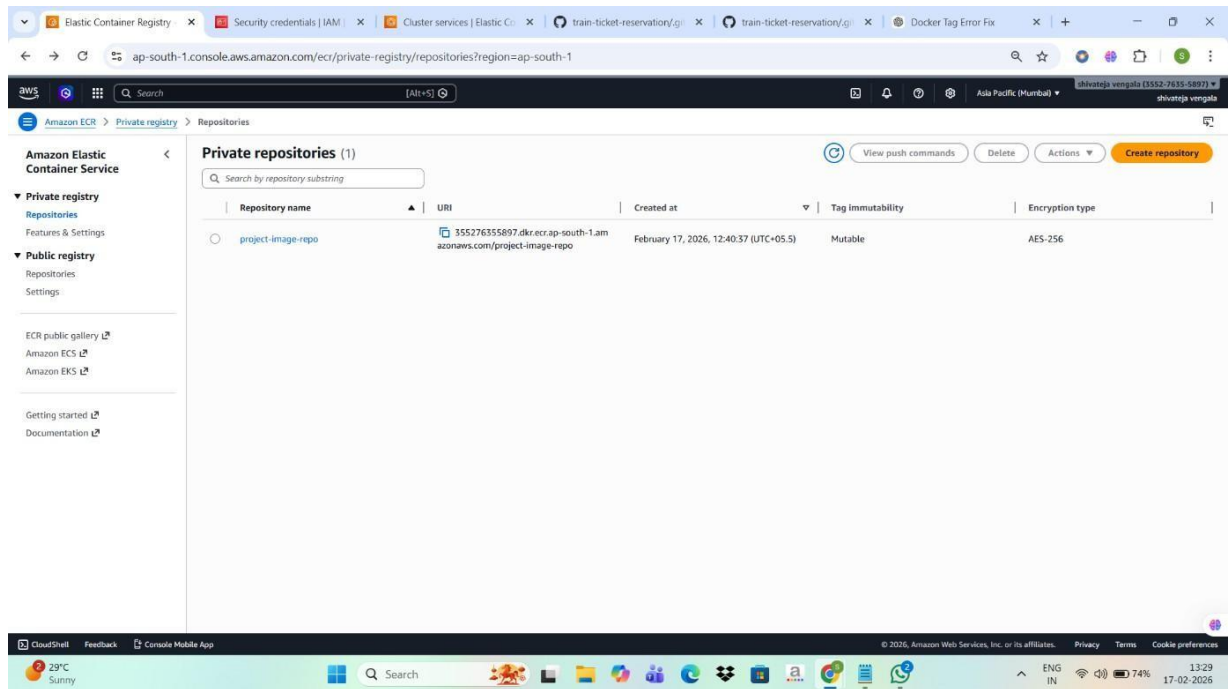
For deployment, **Amazon ECS with Fargate launch type** is used to run the containerized application in a serverless environment. ECS automatically pulls the latest image from ECR and deploys it as a running service, managing infrastructure, scalability, and availability. This automated approach reduces deployment errors, improves system reliability, and increases development productivity. The project demonstrates a practical and industry-relevant DevOps workflow, highlighting the importance of automation, containerization, and cloud-native deployment in modern software engineering.

Deployment Architecture Flow

Code Commit → GitHub Repository
→ GitHub Actions CI/CD Pipeline
→ Docker Image Build
→ Push Image to Amazon ECR
→ Amazon ECS Deployment (Fargate)
→ Running Application Container

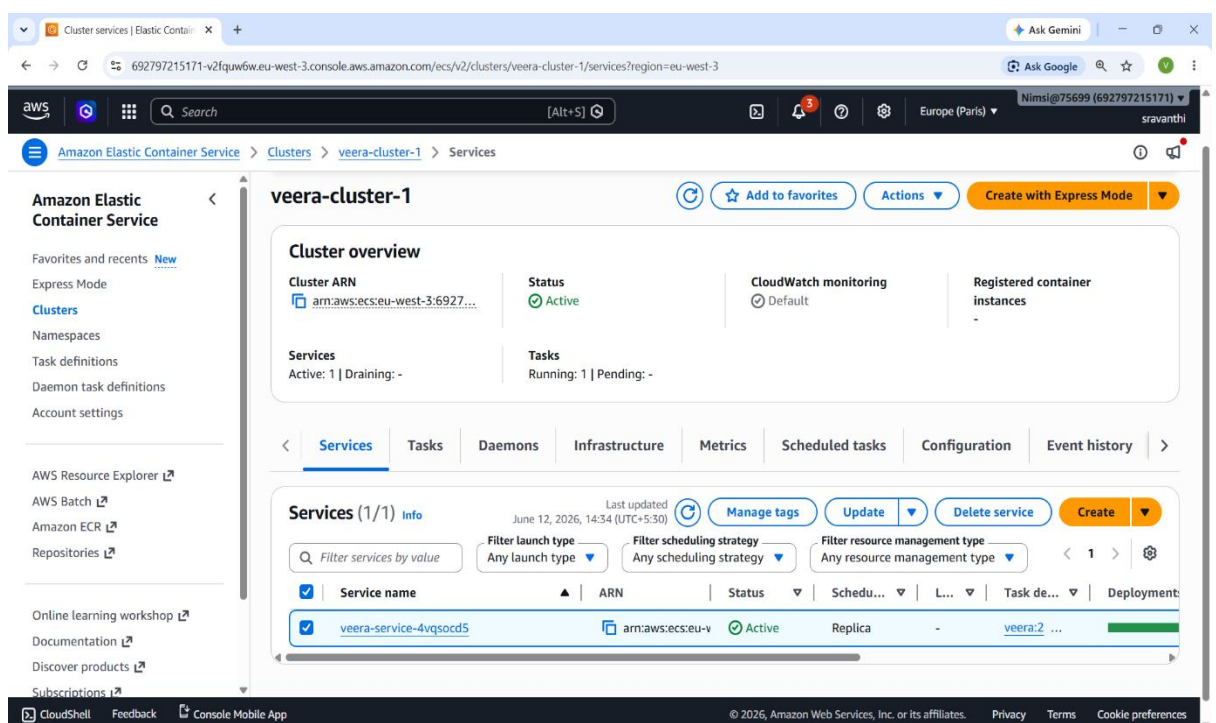
STEP 1: CREATE AMAZON ECR REPOSITORY

- Create a private Docker image repository, Repository name: **“project-image-repo”**
- Used to store application container images.



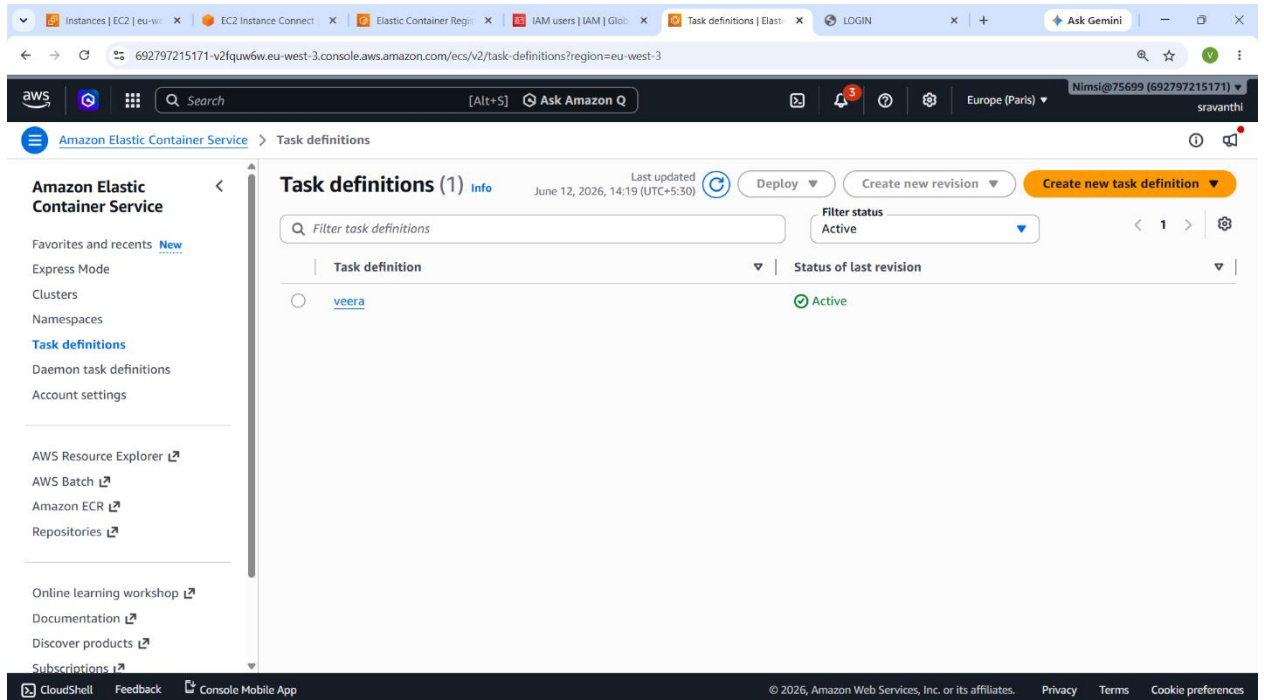
STEP 2: CREATE AMAZON ECS CLUSTER

- Create an ECS cluster using **Fargate launch type**.
- Cluster name: **project-cluster**.



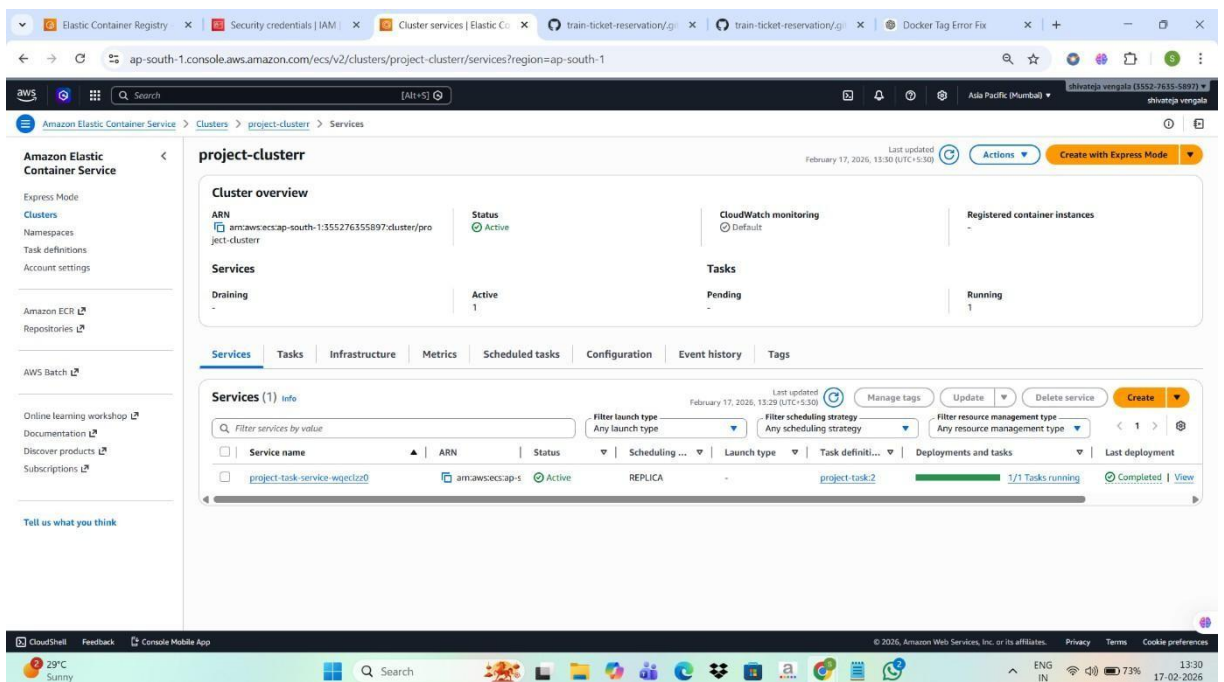
STEP 3: CREATE TASK DEFINITION

- Configure container image from ECR.
- Define CPU and memory allocation.
- Configure port mappings.



STEP 4: CREATE ECS SERVICE

- Create ECS service to maintain the running container.
- Service name: **project-task-service**.



STEP 5: CREATE IAM CREDENTIALS

- Generate **AWS Access Key** and **Secret Key**.

The screenshot shows the AWS IAM console 'IAM users' page. The left sidebar contains navigation links for Identity and Access Management (IAM), Access Management, and Access reports. The main content area shows a list of IAM users. The user 'veera-iam' is selected, and its details are displayed below the list.

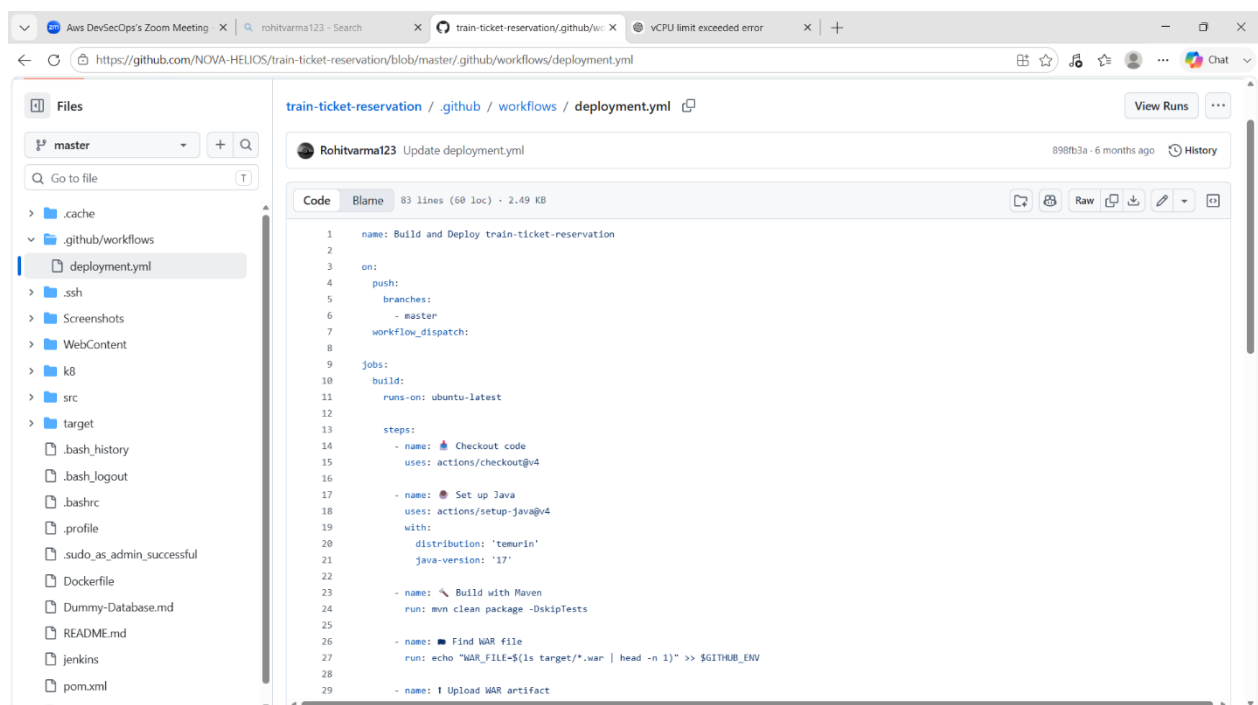
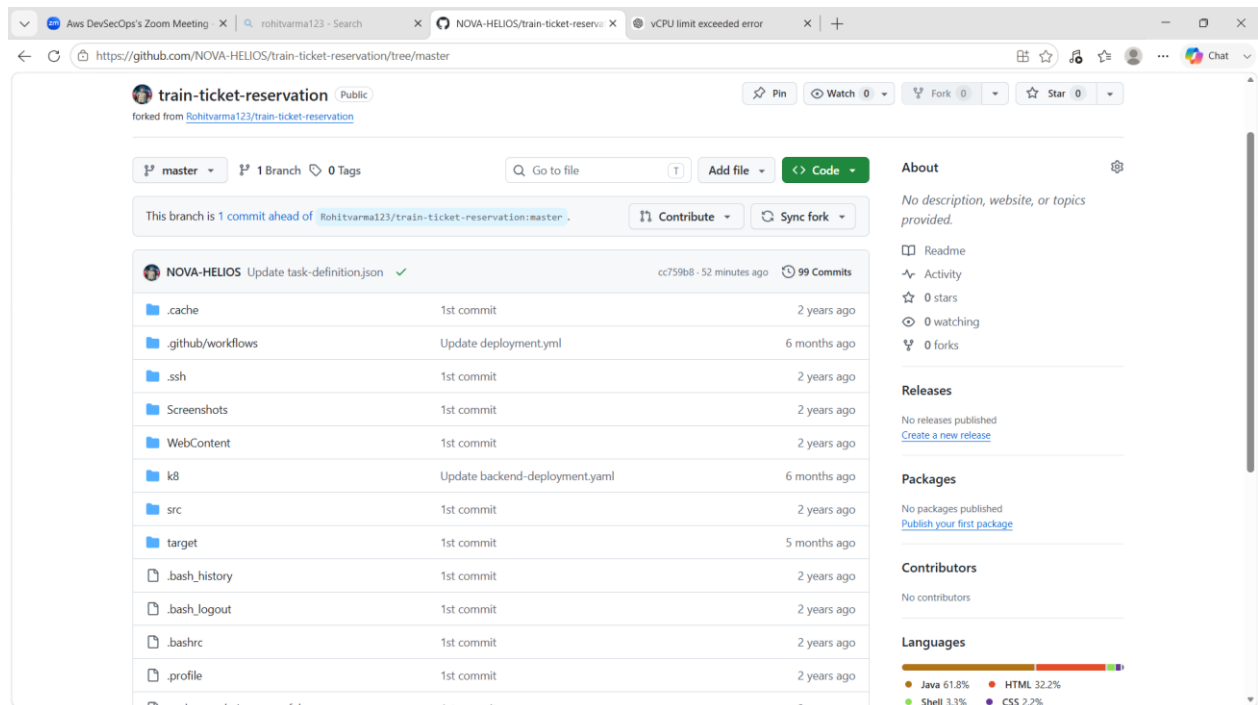
User name	Path	Group	Last activity	MFA	Password age	Console last sign-in	Actions
sravanthi	/	0	2 hours ago	-	31 days	2 hours ago	Act
veera-iam	/	0	36 minutes ago	-	-	-	Act

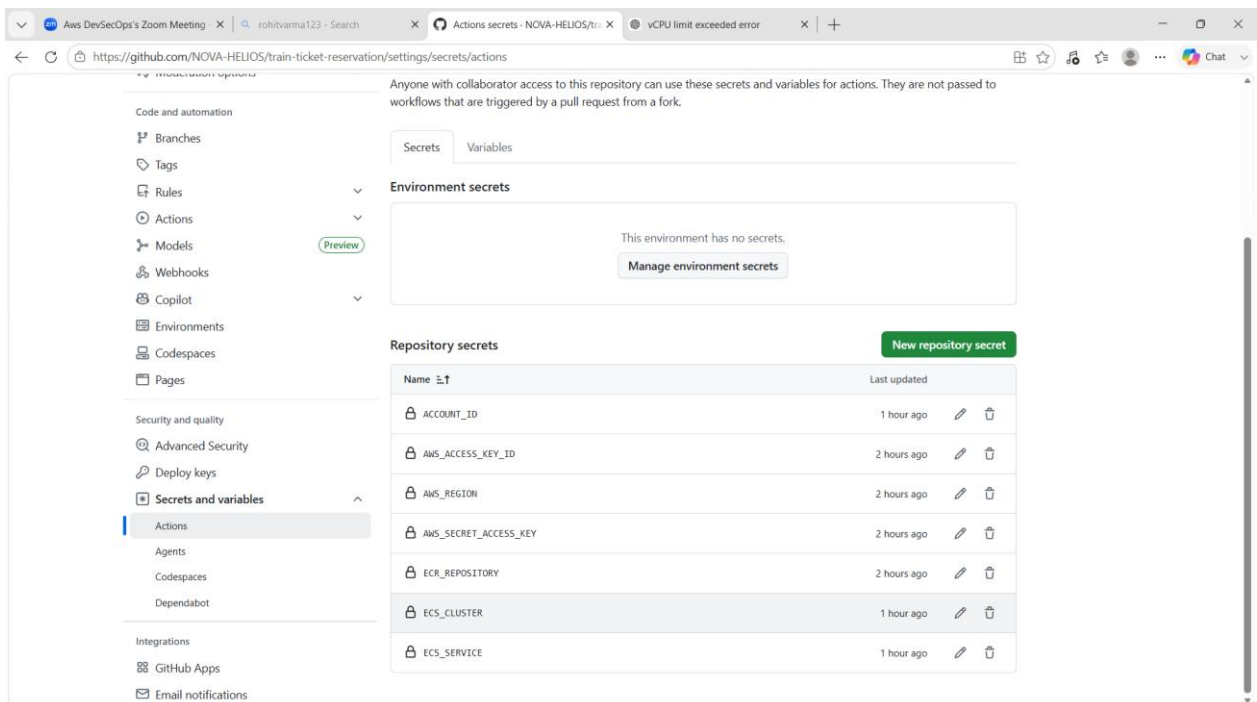
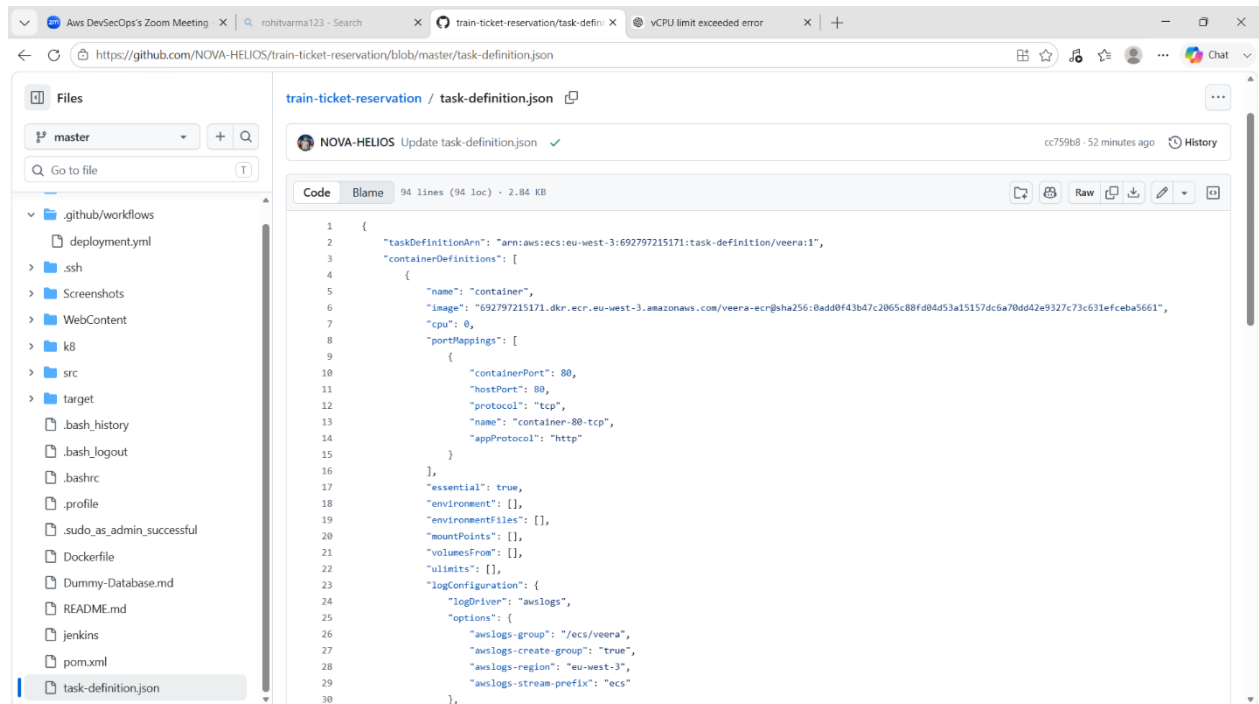
The screenshot shows the AWS Amazon ECR console 'Images' tab for the 'veera-ecr' repository. The left sidebar contains navigation links for Amazon ECR, Private registry, and Public registry. The main content area shows a list of images in the repository.

Image tags	Type	Created at	Image size (MB)	Image digest	Last pulled at
1781251517	Image	June 12, 2026, 13:35:33 (UTC+05.5)	224.38	sha256:9dc3...	-
1781251272	Image	June 12, 2026, 13:31:30 (UTC+05.5)	224.38	sha256:95b...	-
1781250944	Image	June 12, 2026, 13:26:02 (UTC+05.5)	224.38	sha256:9e1...	-
1781248362	Image	June 12, 2026, 12:43:16 (UTC+05.5)	224.38	sha256:0ad...	June 12, 2026, 13:03:18 (UTC+05.5)

STEP 6: CONFIGURE GITHUB REPOSITORY

- Store application source code in GitHub repository.
- Add **Dockerfile** inside the repository.
- Create **.github/workflows/main.yaml** to define CI/CD pipeline.
- Add AWS secrets to GitHub Actions secrets.

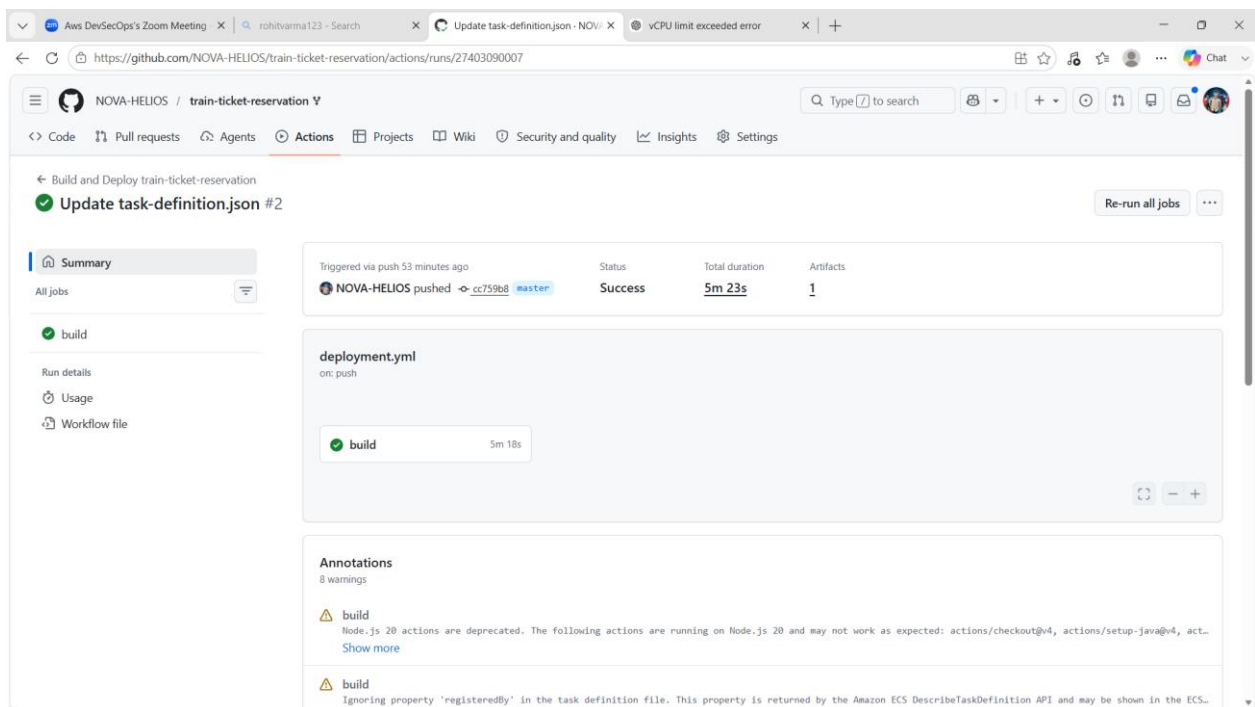




STEP 7: CI/CD PIPELINE USING GITHUB ACTIONS

Pipeline stages include:

- Source code checkout.
- Docker image build.
- Authentication to Amazon ECR.
- Push Docker image to ECR.
- Trigger ECS deployment.
- Run container on AWS Fargate.



STEP 8: DEPLOYMENT VERIFICATION

A) ECS Validation

- Verify ECS service status as **Active**.
- Confirm running task status.

Instances | EC2 | eu-west-3 | EC2 Instance Connect | Elastic Container Registry | IAM users | IAM | Global | Service tasks | Elastic Container Service | LOGIN | Ask Gemini

692797215171-v2fqw6w.eu-west-3.console.aws.amazon.com/ecs/v2/clusters/veera-cluster-1/services/veera-service-4vqsocd5/tasks?region=eu-west-3

Search [Alt+S] Ask Amazon Q Europe (Paris) Nimsi@75699 (692797215171) sravanthi

Amazon Elastic Container Service > Clusters > veera-cluster-1 > Services > veera-service-4vqsocd5 > Tasks

Amazon Elastic Container Service

- Favorites and recents [New](#)
- Express Mode
- Clusters
- Namespaces
- Task definitions
- Daemon task definitions
- Account settings

AWS Resource Explorer [↗](#)

AWS Batch [↗](#)

Amazon ECR [↗](#)

Repositories [↗](#)

Online learning workshop [↗](#)

Documentation [↗](#)

Discover products [↗](#)

Subscriptions [↗](#)

CloudShell Feedback Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Status [Active](#) Tasks (1 Desired) [0 Pending](#) | [1 Running](#) Task definition: revision [veera:2](#) Deployment status [Success](#)

< Health and metrics **Tasks** Logs Deployments Events Configuration and networking Service auto scaling >

Tasks (1/1) Last updated June 12, 2026, 14:26 (UTC+5:30) [Stop](#)

Containers for task be3e17536a48474f8a397b1ad6eecccc

Containers (1)

Filter containers

Container name	Container runtime ID	Image URI	Image Digest	Status	Health status	Containers
container	be3e17536a4847...	69279721...	sha256:0a...	Running	Unknown	0

Instances | EC2 | eu-west-3 | EC2 Instance Connect | Elastic Container Registry | IAM users | IAM | Global | Service health and metrics | LOGIN | Ask Gemini

692797215171-v2fqw6w.eu-west-3.console.aws.amazon.com/ecs/v2/clusters/veera-cluster-1/services/veera-service-4vqsocd5/health?region=eu-west-3

Search [Alt+S] Ask Amazon Q Europe (Paris) Nimsi@75699 (692797215171) sravanthi

Amazon Elastic Container Service > Clusters > veera-cluster-1 > Services > veera-service-4vqsocd5 > Health

Amazon Elastic Container Service

- Favorites and recents [New](#)
- Express Mode
- Clusters
- Namespaces
- Task definitions
- Daemon task definitions
- Account settings

AWS Resource Explorer [↗](#)

AWS Batch [↗](#)

Amazon ECR [↗](#)

Repositories [↗](#)

Online learning workshop [↗](#)

Documentation [↗](#)

Discover products [↗](#)

Subscriptions [↗](#)

CloudShell Feedback Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

veera-service-4vqsocd5 [Info](#) [Add to favorites](#) [Delete service](#) [Update service](#)

Service overview [Info](#)

Status [Active](#) Tasks (1 Desired) [0 Pending](#) | [1 Running](#) Task definition: revision [veera:2](#) Deployment status [Success](#)

< **Health and metrics** Tasks Logs Deployments Events Configuration and networking Service auto scaling >

Status [Info](#)

Service name [veera-service-4vqsocd5](#) Service ARN [arn:aws:ecs:eu-west-3:6...](#) Deployments current state [1 Completed task](#) Created at [June 12, 2026, 13:20 \(UTC+5:30\)](#)

Health check grace period 0 seconds

Health

☐ Alarm recommendations [3h](#) [1d](#) [1w](#) [UTC timezone](#) [Explore related](#)

Instances | EC2 | eu-west-3 | EC2 Instance Connect | Elastic Container Registry | IAM users | IAM | Global Access | Cluster tasks | Elastic Container Service | LOGIN | Ask Gemini

692797215171-v2fqw6w.eu-west-3.console.aws.amazon.com/ecs/v2/clusters/veera-cluster-1/tasks?region=eu-west-3

Search [Alt+S] Ask Amazon Q Europe (Paris) Nimsi@75699 (692797215171) sravanthi

Amazon Elastic Container Service

Cluster overview

Cluster ARN: [arn:aws:ecs:eu-west-3:6927...](#) Status: Active CloudWatch monitoring: Default Registered container instances: -

Services: Active: 1 | Draining: - Tasks: Running: 1 | Pending: -

Services | **Tasks** | Daemons | Infrastructure | Metrics | Scheduled tasks | Configuration | Event history

Tasks (2)

Last updated: June 12, 2026, 14:18 (UTC+5:30) Manage tags Stop Run new task

Filter tasks by property or value Filter desired status: Any desired status Filter launch type: Any launch type

Task	Last status	Desired status	Task definition	Health status	Start
be3e17536a48474f8a397...	Running	Running	veera:2	Unknown	ecs-s
c9c03e92dea04d4c869c1...	Stopped	Stopped	veera:1	Unknown	ecs-s

CloudShell Feedback Console Mobile App © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Instances | EC2 | eu-west-3 | EC2 Instance Connect | Elastic Container Registry | IAM users | IAM | Global Access | Clusters | Elastic Container Service | LOGIN | Ask Gemini

692797215171-v2fqw6w.eu-west-3.console.aws.amazon.com/ecs/v2/clusters?region=eu-west-3

Search [Alt+S] Ask Amazon Q Europe (Paris) Nimsi@75699 (692797215171) sravanthi

Amazon Elastic Container Service

veera-service-4vqsocd5 has been deployed successfully. View service

Notifications 0 0 7 0 0 0

Clusters (1) Info Create cluster

Search clusters

By default, we only load up to 1,000 clusters at a time.

Cluster	Services	Tasks	Container instances	CloudWatch monitoring
veera-cluster-1	1	0 Pending 1 Running	0 EC2	Default

CloudShell Feedback Console Mobile App © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Application Testing

- Retrieve public IP from ECS task.
- Access the application through a web browser.

The screenshot shows the AWS Management Console 'Instances' page. The instance 'veera-instance' (i-0daa5ae7ddc6d2383) is in a 'Running' state. The public IPv4 address is 35.180.63.211. The instance type is t2.micro and it is located in the eu-west-3 region.

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4 ...
veera-instance	i-0daa5ae7ddc6d2383	Running	t2.micro	2/2 checks passed	eu-west-3c	ec2-35-180-63-211.eu-...	35.180.63.211

i-0daa5ae7ddc6d2383 (veera-instance)

Instance summary

Instance ID	Public IPv4 address	Private IPv4 addresses
i-0daa5ae7ddc6d2383	35.180.63.211 open address	172.31.32.114

Instance state

Running

Public DNS

ec2-35-180-63-211.eu-west-3.compute.amazonaws.com | open

The screenshot shows the AWS CloudShell terminal. The user is running 'docker run' commands to start a container. The terminal output shows errors related to container networking and port conflicts. The instance 'veera-instance' (i-0daa5ae7ddc6d2383) is shown with public IP 35.180.63.211 and private IP 172.31.32.114.

```
See 'docker run --help' for more information
root@ip-172-31-32-114:~# docker run --name be3e17536a48474f8a397blad6ecccc0-2377118072 -d -p 8080:8080 0add0f43b47c
ce4ceb4425fdb4e77de99627e318bf58ae3f1906f3f86216fd99d698825d1046
docker: Error response from daemon: failed to set up container networking: driver failed programming external connectivity on endpoint be3e17536a48474f8a397blad6ecccc0-2377118072 (ebf5f71a124dbd2b4c704de03099f3d3772c575abce1232c77e9c0993d52dcf6): Bind for 0.0.0.0:8080 failed: port is already allocated

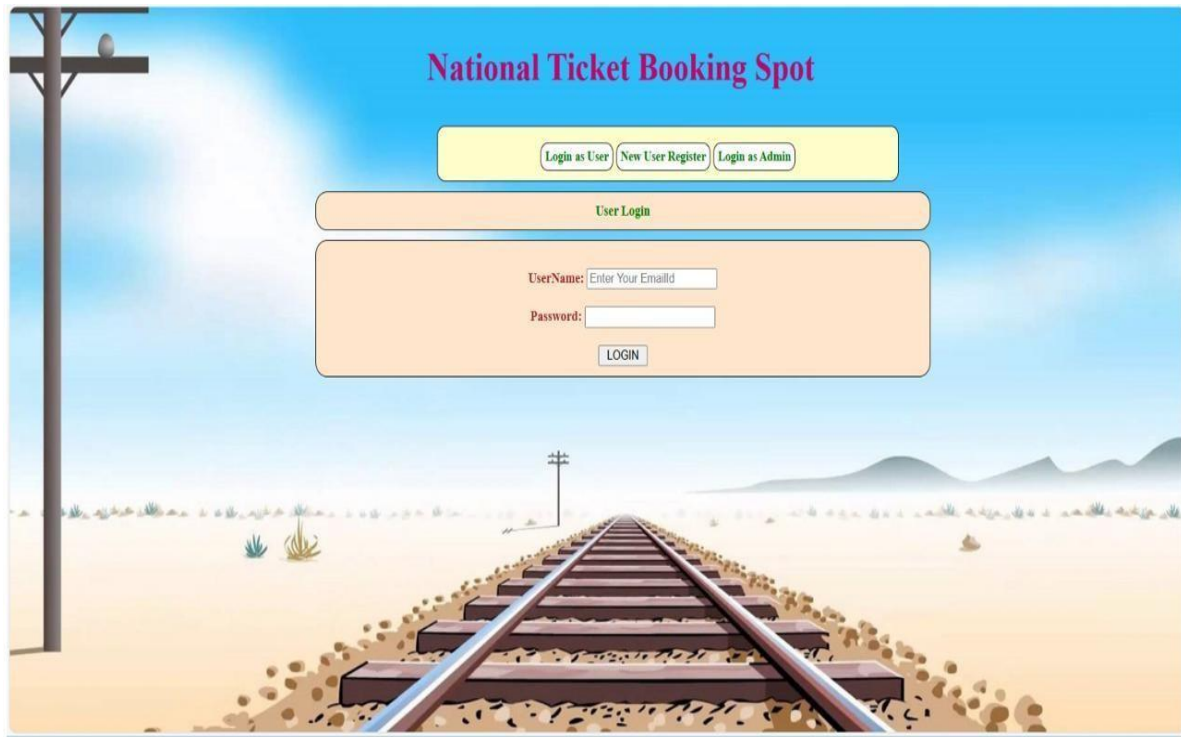
Run 'docker run --help' for more information
root@ip-172-31-32-114:~# docker run --name be3e17536a48474f8a397blad6ecccc0-2377118072 -d -p 8085:8080 0add0f43b47c
47c
docker: Error response from daemon: Conflict. The container name "/be3e17536a48474f8a397blad6ecccc0-2377118072" is already in use by container "ce4ceb4425fdb4e77de99627e318bf58ae3f1906f3f86216fd99d698825d1046". You have to remove (or rename) that container to be able to reuse that name.

Run 'docker run --help' for more information
root@ip-172-31-32-114:~# docker ps
CONTAINER ID        IMAGE                                     COMMAND                  CREATED              STATUS              PORTS
408da2f96a99        692797215171.dkr.ecr.eu-west-3.amazona... "catalina.sh run"       59 minutes ago      Up 59 minutes      0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
veera-app
root@ip-172-31-32-114:~# docker run --name be3e17536a48474f8a397blad6ecccc0-2377118072 -d -p 8085:8080 0add0f43b47c
47c
docker: Error response from daemon: Conflict. The container name "/be3e17536a48474f8a397blad6ecccc0-2377118072" is already in use by container "ce4ceb4425fdb4e77de99627e318bf58ae3f1906f3f86216fd99d698825d1046". You have to remove (or rename) that container to be able to reuse that name.

Run 'docker run --help' for more information
root@ip-172-31-32-114:~# docker ps
CONTAINER ID        IMAGE                                     COMMAND                  CREATED              STATUS              PORTS
408da2f96a99        692797215171.dkr.ecr.eu-west-3.amazona... "catalina.sh run"       About an hour ago    Up About an hour    0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
veera-app
root@ip-172-31-32-114:~#
```

i-0daa5ae7ddc6d2383 (veera-instance)

PublicIPs: 35.180.63.211 PrivateIPs: 172.31.32.114



B) Image Verification

- Verify updated Docker image version in ECS task details.

Instances | EC2 | eu-west-3 | EC2 Instance Connect | Elastic Container Registry | Create access key | IAM | Service tasks | Elastic C | LOGIN | Ask Gemini

692797215171-v2lquw6w.eu-west-3.console.aws.amazon.com/ecr/private-registry/repositories?region=eu-west-3

aws Search [Alt+S] Europe (Paris) Nimsil@75699 (692797215171) sravanthi

Amazon ECR > Private registry > Repositories

Amazon Elastic Container Registry

▼ Private registry

- Repositories
- Features & Settings

▼ Public registry

- Repositories
- Settings

ECR public gallery [↗](#)

Amazon ECS [↗](#)

Amazon EKS [↗](#)

Getting started [↗](#)

Documentation [↗](#)

Private repositories (1)

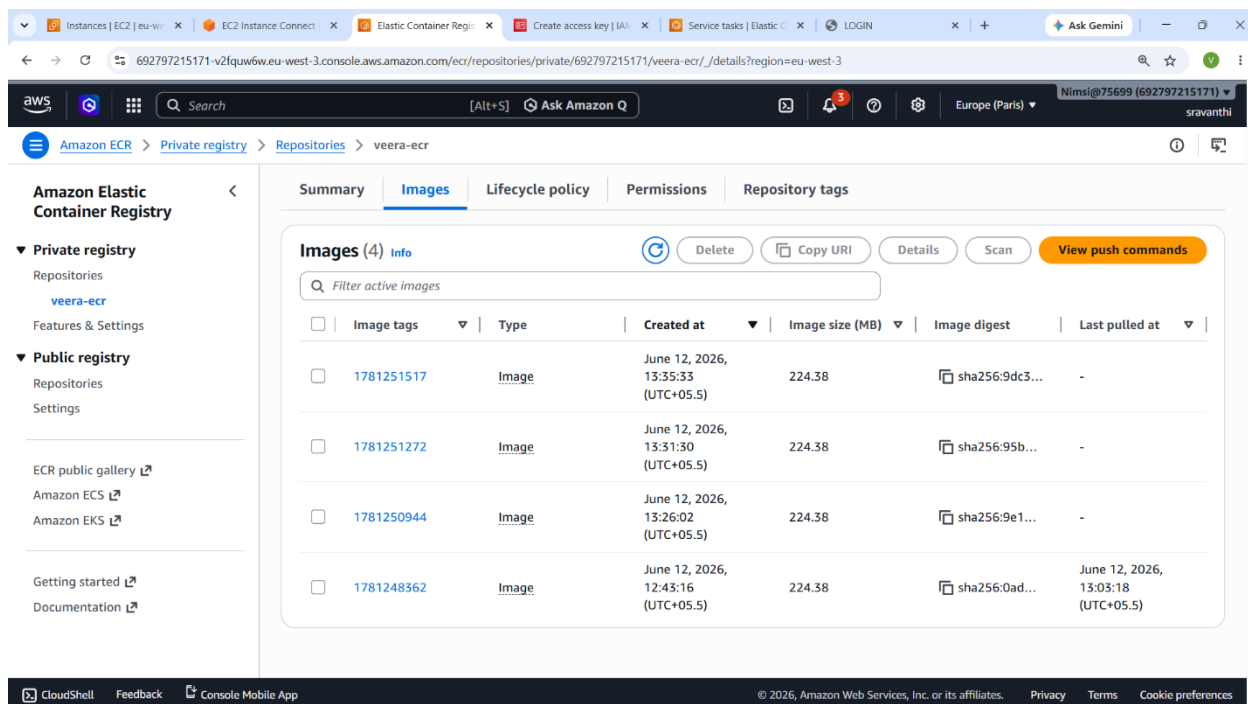
[View push commands](#) [Delete](#) [Actions](#) [Create repository](#)

Search by repository substring

Repository name	URI	Created at	Tag immutability	Encryption type
☐ veera-ecr	692797215171.dkr.ecr.eu-west-3.amazonaws.com/veera-ecr	June 12, 2026, 11:53:01 (UTC+05.5)	Mutable	AES-256

CloudShell Feedback Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences



CONCLUSION

This project successfully demonstrates a fully automated CI/CD pipeline for deploying a Java-based Train Ticket Application using modern DevOps tools and AWS cloud services. The integration of **GitHub Actions**, **Docker**, **Amazon ECR**, and **Amazon ECS Fargate** transformed the deployment process into an efficient, scalable, and reliable system.

Containerization ensured application consistency across environments, while GitHub Actions enabled continuous integration and automated deployments. Amazon ECR provided secure image storage, and ECS Fargate eliminated the need for server management by enabling serverless container execution.

The implemented pipeline significantly reduced deployment time, minimized human errors, and improved overall application reliability. This project reflects real-world DevOps workflows used in enterprise environments and provides a strong foundation for future enhancements such as auto-scaling, load balancers, blue-green deployments, and advanced monitoring.

Overall, this implementation highlights the importance of CI/CD automation and cloud-native technologies in building modern, production-ready applications.

